6

Beyond Markovian Dynamics

> "when life reaches a certain
> temperature, our souls and our blood
> . . . live at ease in contradiction, as
> indifferent to duty as to faith"
>
> Albert Camus, *Personal Writings*

Most modern machine learning and statistical inference methods treat the learner as a passive observer estimating properties of a fixed world. Even though Reinforcement Learning involves an active agent, most frameworks assume the agent's knowledge and actions can be cleanly separated: The agent observes a state, chooses an action, and then receives a reward as if each step were independent. But what happens when an agent's *beliefs* and *actions* become entangled? The question science asks, according to Feynman [49], is "If I do this, what will happen?" - so how does one learn what will happen when what you can know depends on what you do, and if what you should do depends on what you can know? Such occurrences arise in classic Reinforcement Learning frameworks when the Markov property is violated. Causal Inference techniques can augment Reinforcement Learning methods to optimize systems with non-Markovian dynamics.

When the Markov property is violated for any reason, the agent's state representation no longer suffices to render future rewards conditionally independent of the past. In such settings, nonlinearity arises as the product of a mismatch between the assumed decision process and the true data-generating mechanism. We will see in future chapters that these failures arise from non-ignorability: Markovian dynamics are lost when the policy is driven by unobserved determinants of reward that cannot be determined from the observed state.

6.1 When History Matters: Violations of the Markov Property

So far, every Reinforcement Learning algorithm we have discussed relies on the Markov property: We assume that the state transition probabilities depend only on the current state, which is fully observed. However, the Markov property does not always hold in real-world scenarios. Markov violations commonly arise from unobserved state components (partial observability) or true temporal dependencies (history dependence). Causal methods can mitigate these difficulties.

Carryover effects, commonly encountered in a type of clinical trial called the *crossover trial*, can cause the Markov property to fail. In the crossover design, patients are initially randomized to a treatment, and then after a while they are switched to the other treatment. Each patient's outcome on drug 2 is compared to the same patient's outcome on drug 1, thereby eliminating the between-subject variability which might obscure results.

The carryover effect happens here when effects from the first drug taken affect the

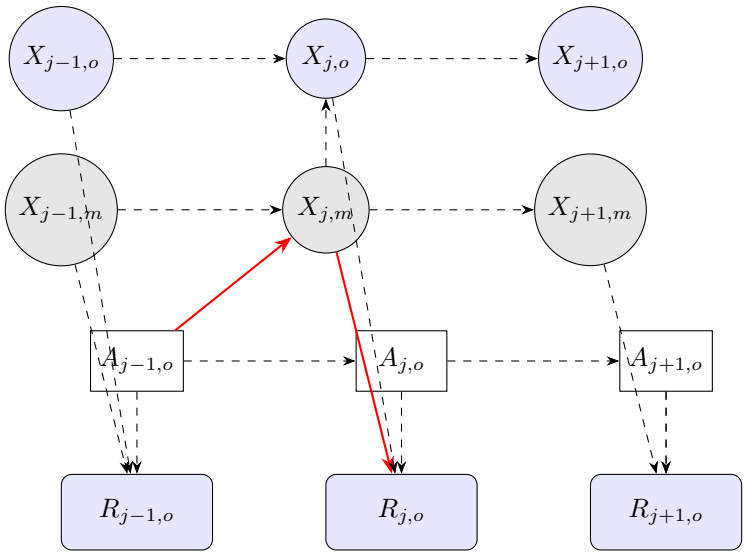

FIGURE 6.1: Diagram of a confounded MDP with nonlinearity path highlighted in red.

patient's outcome in the second period. For example, suppose the first drug taken has a delayed effect. Then, this delayed efficacy could be falsely attributed to the second drug taken: Here, the carry-over from the first drug results in a violation of the Markov property: $\Gamma(x_{j+1}|x_{j-k:j}, a_{j-k:j}) \neq \Gamma(x_{j+1}|x_j, a_j)$.

Carryover effects create time-dependent confounding: The previous treatment a_{j-1} affects the current state, which both influences the current outcome and the current treatment selection, thereby inducing a temporal dependency similar to the one depicted in Figure 3.3.

One might interpret this temporal dependency in the data as a missingness problem, as depicted in Figure 6.1. Here, the unobserved part of the state corresponds to the amount of drug A left in the subject's system at iteration j, assuming the subject switched to drug B at time $j' < j$; the problem is that the latent drug concentration is non-ignorably missing. When ignorability assumptions are violated, our classic RL algorithms can converge to suboptimal policies - we will return to non-ignorable missing data in the following chapter.

Consider the following application to intensive care medicine. Dr. Yersin is treating Marcus, an elderly patient who has developed sepsis following surgery. Marcus's condition is deteriorating rapidly, and Dr. Yersin must decide between several treatment strategies: She could take action $a^{(1)}$ and administer high-dose antibiotics immediately, or $a^{(2)}$ and start with a more conservative antibiotic regimen, or choose $a^{(3)}$ and focus first on fluid resuscitation to stabilize his blood pressure.

The challenge is that Dr. Yersin cannot directly observe Marcus's true physiological state X_j. She has access to vital signs from monitors $X_{j,o}$, but these are noisy and delayed. Marcus' latest blood work is from three hours ago, and the results of the cultures that would definitively identify the bacterial strain won't be available for another day. His kidney function appears compromised, but it is unclear whether this is due to the sepsis itself, dehydration, or a reaction to medications he received earlier.

Suppose Dr. Yersin chooses the aggressive antibiotic approach at time t, reasoning that the potential benefits outweigh the risks. Marcus's blood pressure stabilizes over the next few hours, and his white blood cell count begins to improve. However, by time $t + 2$, his kidney function has worsened significantly.

Is the kidney deterioration a delayed effect of the sepsis that was already in progress

(meaning Dr. Yersin's aggressive treatment was correct), or is it a side effect of the high-dose antibiotics she administered (meaning she should have chosen the conservative approach)? The answer depends on Marcus's true underlying physiological state $X_{j,m}$ at time t, which was never fully observable. While the true state sequence $\{X_j = (X_{j,o}, X_{j,m})\}_{j=0}^{\infty}$ may well be Markovian, the observed state components $X_{j,o}$ alone are insufficient for optimal decision-making, and may appear to behave in a non-linear fashion. Temporal violations such as these can be addressed using the marginal structural modeling approach introduced in Section 3.4.

Recall that in the crossover trial, the problem is that previous treatments a_{j-1} affect current states x_j, which then influence both current outcomes and treatment selection. The carryover effect creates the time-dependent confounding structure $A_{j-1} \to X_j \to R_j$, where $X_j \to A_j$, as pictured in Figure 3.3. Following the standard Causal Inference methodology described in Chapter 3, one might resolve this issue by creating a pseudo-population through sequential reweighting. The stabilized weights from Equation 3.7 effectively break the confounding path by adjusting for the fact that treatment at time t' depends on the confounder $\bar{x}_{it'}$, which itself was affected by previous treatments [133].

In the case of Marcus's sepsis treatment, if we observe that aggressive antibiotic use ($a_j = a^{(1)}$) at time j is followed by kidney deterioration at $j+2$, then standard RL methods would, potentially incorrectly, update $Q(x_j, a^{(1)})$ downward. But the kidney deterioration might be due to the sepsis state at $j-1$, not the antibiotic choice.

6.2 Partially Observed Markov Decision Processes

We have discussed how one might use the MSM to address violations of the Markov property. In our sepsis example, the MSM weights recognize that Dr. Yersin's choice of aggressive treatment was influenced by Marcus's deteriorating condition, and account for it by adjusting the treatment estimates based on $\bar{x}_{it}$. The weight ω_{ij} upweights patients who received Dr. Yersin's rare aggressive treatment despite appearing stable, and downweights those more common instances of patients who received aggressive treatment while deteriorating. But what if we lack sufficient data to fit an MSM? Suppose Marcus is the first to receive Dr. Yersin's rare treatment. In this case, one might instead use an extension of the MDP called a *Partially Observed Markov Decision Process*, which allows state components to be unobserved. These unobserved state components are interpreted as random variables; at each MDP iteration, they are estimated using Bayes' formula.

A POMDP is visualized in Figure 6.2. Here, the missing information x_m might represent unobserved patient characteristics that influence Dr. Yersin's treatment decisions. The belief states b fill the role of the MSM weights by conditioning on these unobserved variables.

Unlike MSMs, however, POMDPs reweight through forward belief propagation, computing a sum over the x_m weighted by their corresponding probability distributions. Conversely, MSMs achieve the same result through a type of "backward" belief propagation, i.e. inverse probability weighting: MSMs *measure* the latent information encoded in the treatment frequencies, while POMDPs *model* the latent variable. Note that the former strategy works only insofar as the latent state leaves fingerprints in the observed history: If treatment assignment depends on x_m through channels that never register in x_o, then no reweighting of the observed data can recover the confounded effect.

Consider, for example, a simplified crossover trial where patients receive two treatments sequentially. In this trial, the observed state $X_{j,o}$ takes values in {Normal BP, Low BP}, the hidden state $X_{j,m}$ takes values in {No residual drug, Residual drug}, the action A_j takes

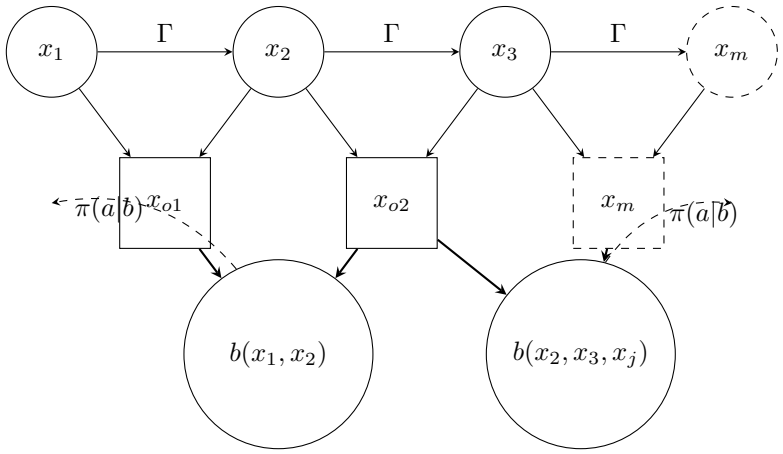

FIGURE 6.2: Visualization of a POMDP. States x transition according to dynamics Γ, but the agent only observes partial information x_o (with some components x_m missing). The agent maintains belief states $b(x)$ representing probability distributions over possible underlying states.

values in {Drug A, Drug B}, and the outcome is blood pressure reduction R_j.

The hidden residual drug concentration affects both future observations and optimal treatment in two ways. First, if residual Drug A is present and we give Drug B, there is a dangerous interaction. Second, residual drug presence lowers BP, which confounds the observation.

Using POMDP belief tracking, at each time j, we maintain a set of beliefs $b_j = [P(\text{No residual}), P(\text{Residual})]$. Starting from uniform prior $b_0 = [0.5, 0.5]$, after observing Drug A followed by Normal BP, we update to $b_1 = [0.3, 0.7]$ (likely has residual). After observing Drug B followed by Low BP, we update to $b_2 = [0.1, 0.9]$ (strong residual effect). Then, we can compute the belief-weighted value estimation as in 6.1.

$$V^{\pi}(x_o) = 0.1 \cdot V(\text{Low BP, No residual}) + 0.9 \cdot V(\text{Low BP, Residual}) \tag{6.1}$$

The formal update rule (which implements Bayes' theorem), is expressed in 6.2, which marginalizes over the distribution of each latent node in 6.2.

$$b_{j+1}(x') = \frac{P(X_{j,o} = x_o | X_j = x', A_{j-1} = a) \sum_{x \in \Omega} \Gamma(x'|x, a) b_j(x)}{\sum_{x'' \in \Omega} P(X_{j,o} = x_o | X_j = x'', A_{j-1} = a) \sum_{x \in \Omega} \Gamma(x''|x, a) b_j(x)} \tag{6.2}$$

For comparison, consider the MSM strategy for the same trajectory, where we compute stabilized weights. If the marginal probability of Drug B is 0.5 but the conditional probability given Low BP is 0.8, then the stabilized weight can be computed as in 6.3, downweighting the observation because Drug B was expected given the Low BP (which was itself caused by residual Drug A). Like the POMDP strategy, the weighted outcome correctly attributes the effect to the residual drug rather than Drug B.

$$\omega_2 = \frac{P(A_2 = \text{Drug B})}{P(A_2 = \text{Drug B} | \text{Low BP})} = \frac{0.5}{0.8} = 0.625 \tag{6.3}$$

Note that both methods ideally achieve the same deconfounding, though the estimates are not necessarily exactly identical. The POMDP explicitly tracks the probability of residual drug (0.9), while the MSM implicitly accounts for it through the weight (0.625): The

structure of a POMDP can be represented as a dynamic causal model where hidden states $X_{j,m}$ act as time-varying confounders affecting both the observable state components $X_{j,o}$ and the rewards r_j (under this conceptualization, the confounding structure $X_{j,m} \to X_{j,o}$, $X_{j,m} \to r_j$ arises, where the hidden state is a common cause). Similarly, the belief update can be interpreted as a sequential application of Pearl's do-calculus [117]: We compute $P(X_{j+1}|\pi(A_j = a_j) = 1, X_{j,o})$ by marginalizing over the unobserved confounder $X_{j,m}$. The belief state $b_j(x)$ serves as our running estimate of the confounder distribution, updated via Bayes' rule as new evidence arrives.

Historical Note: The synergy between Bayesian methods and Reinforcement Learning is not coincidental. Cornfield, one of the early founding researchers of Bayesianism, was likely influenced at least indirectly by Bellman's work; Cornfield's close collaborator L. Savage [52, 144, 146] had a brother, I.R. Savage, who was cited in Richard Bellman's initial 1957 paper on Dynamic Programming [13]. Moreover, both Savage brothers as well as Bellman and Cornfield all attended a scientific meeting in Washington in 1953 [1].

There is a connection through all this to the Causal Inference space as well. Recall from Chapter 2 the statistician *Donald Rubin*, who invented the potential outcomes framework. Rubin's advisor William Cochran served on the 1964 Surgeon General's Advisory Committee, which explicitly built its causal methodology on Cornfield's 1959 framework for assessing smoking and lung cancer. Cochran and Rubin collaborated on observational study methods [36] before Rubin formalized the potential outcomes framework [136]. Rubin later presented at the Jerome Cornfield Memorial Session in 1980 [66], acknowledging Cornfield's foundational contributions to Causal Inference. Rubin's student Daniel F. Heitjan subsequently expanded the non-ignorability framework and showed its various applications in clinical data; more recently, Heitjan's student MaryLena Bleile (that's me) developed a relativistic extension of the ignorability criterion which can be used to guarantee convergence of Q-learning in cases where the standard convergence theorems do not apply [21] (though I was actually unaware of this intellectual lineage when I did so). We will return to relative ignorability in Chapter 7.

Recent research has confirmed the connection between causality and Reinforcement Learning [150], showing that online Reinforcement Learning inherently performs Causal Inference when the environment contains unobserved confounders, with belief states representing probability distributions over latent causal variables. Survey work on Causal Reinforcement Learning [188] has also demonstrated that POMDPs with confounded transitions can be represented using structural causal models, enabling counterfactual reasoning about alternative policies.

What, then, is the practical difference between POMDP and MSM approaches to confounding? We have established that the POMDP assumes a probability distribution over the latent variables, while MSMs estimate the action-reweighting probability terms directly. As a result, POMDPs are more costly to fit; belief updates require $O(|\Omega|^2 \cdot |A|)$ operations per update and need models for both $P(x_o|x)$ and $\Gamma(x'|x, a)$, since they maintain a full distribution over x_m. In contrast, MSMs are most suitable for batch analysis and shorter trajectories; the MSM approach requires only $O(1)$ operations per weight computation and needs only the propensity score model $\pi(a|x_o)$ to maintain its scalar weights. Typically POMDPs are more robust if correctly specified, and most suitable for online planning. However, they are sensitive to model misspecification, and typically more data-intensive to specify and fit. Note, however, that the two methods demand different *kinds* of data: The MSM pools information across *patients*, requiring breadth (many patients, including some

who actually received each treatment of interest), while the POMDP accumulates information across *time*, requiring depth (many observations within a single trajectory)[1]. If nobody before Marcus has ever received the rare treatment, there are no replicates for the MSM to reweight - however, the POMDP can still evaluate the novel action through its transition model, leaning on modeling assumptions precisely where data are absent and refining its belief state online as Marcus's own responses accumulate. Doubly robust methods combine both approaches, achieving both unbiasedness and variance reduction compared to standard importance sampling estimators [74, 167]. We will return to these in section 6.3.

In practice, whether the MSM or the POMDP is ideal for one's problem ultimately depends on one's computational resources and modeling assumptions. If you can specify accurate transition and observation models and need to plan adaptively, use the POMDP framework. If you have good propensity score estimates and are analyzing batch data, MSM weights are more efficient. When model specification is uncertain, doubly robust methods that combine both approaches provide additional protection against mis-specification.

6.3 Doubly Robust Estimation Methods

In the above example, we assume that Dr. Yersin is actively treating Marcus; Dr. Yersin has the capacity to actively choose treatments according to the desired policy π. But this active choice of actions is not always possible, as emphasized in our previous discussion of offline Q-learning methods. Often, we may need to evaluate a target policy π using data generated by a different behavior policy π_b.

This situation is particularly challenging when we cannot observe all of the data: Unobserved state components X_m affect both the actions taken and the rewards received. Moreover, we cannot use the standard importance sampling estimator, since each weight $\omega_j = \prod_{k=j}^{T-1} \frac{\pi(a_k|X_k)}{\pi_b(a_k|X_k)}$ requires that we observe the full state $X = (X_o, X_m)$. When X_m is unobserved, we cannot directly compute either the numerator or the denominator of this ratio.

The doubly robust estimator developed in [74] addresses this problem by combining a direct outcome model with importance weighting. For a single timestep, the doubly robust estimator of the value function takes the form shown in 6.4, where $\hat{\omega}_j$ are estimated importance weights based on observed covariates x_o, and $\hat{Q}(x_o, a)$ is an estimated Q-function that marginalizes over the unobserved components.

$$\hat{V}_\pi^{DR}(x_o) = \frac{1}{n} \sum_{j=1}^{n} \left[\hat{\omega}_j \cdot (r_j - \hat{Q}(x_{o,j}, a_j)) + \sum_{a \in \mathcal{A}} \pi(a|x_{o,j}) \hat{Q}(x_{o,j}, a) \right] \tag{6.4}$$

Notably, doubly robust Q-learning requires correctness of only *one* of the two estimators used; $\hat{Q}$ or $\hat{\omega}$ (hence the "double" in "doubly robust"). If the Q-function model $\hat{Q}$ is correctly specified, then the bias from incorrect weights $\hat{\omega}_j$ vanishes asymptotically. Conversely, if the propensity score model that generates $\hat{\omega}_j$ is correct, then errors in $\hat{Q}$ cancel out (See

[1] Notice here how the wide-long correspondence of MSM/POMDPs reflects the wide/long duality of R (used in Causal Inference to implement MSMs) vs Python (used in Reinforcement Learning to implement POMDPs): In R, one tends to *widen* each line of code, making the computation more efficient through vectorization, while Python realizes the recursive structure of the Bellman update as explicit iteration, lengthening the script down the page. (see Table 1.1).

Equation 6.5).

$$\mathbb{E}[\hat{V}_\pi^{DR}(x_o)] - V_\pi(x_o) = \underbrace{\mathrm{Cov}(\hat{\omega}_j, \hat{Q} - Q)}_{\text{vanishes if either model correct}} + \underbrace{\mathbb{E}[(\hat{\omega}_j - \omega_j)(\hat{Q} - Q)]}_{\text{higher-order term}} \tag{6.5}$$

For episodic Reinforcement Learning with horizon T, we can construct a doubly robust estimator that combines forward model predictions with backward importance weighting. Let $\hat{\Gamma}$ denote an estimated transition model and define the model-based value estimate per 6.6.

$$\hat{V}_\pi^{\text{model}}(x_{o,j}) = \mathbb{E}_{\hat{\Gamma},\pi}\left[\sum_{k=j}^{T} \gamma^{k-j} \rho(x_{o,k+1}) \mid x_{o,j}\right] \tag{6.6}$$

The sequential doubly robust estimator then takes the form in 6.7, where $\hat{r}_{k+1}$ is the model's prediction of the reward at step $k+1$, and the product of importance weights corrects for the distributional shift between the behavior and target policies.

$$\hat{V}_\pi^{DR}(x_{o,j}) = \hat{V}_\pi^{\text{model}}(x_{o,j}) + \sum_{k=j}^{T} \gamma^{k-j} \left(\prod_{t=j}^{k} \hat{\omega}_t\right) (r_{k+1} - \hat{r}_{k+1}) \tag{6.7}$$

The model-based component $\hat{V}_\pi^{\text{model}}$ plays a role analogous to the POMDP belief update, maintaining predictions about future trajectories under partial observability. Meanwhile, the importance-weighted correction term serves the same deconfounding purpose as MSM weights, adjusting for selection bias in the observed actions. The doubly robust estimator effectively runs both procedures in parallel, using each to protect against failures in the other.

In the practical deployment of doubly robust methods, one must pay careful attention to variance control. While the doubly robust estimator is consistent under weaker conditions than either approach alone, we are using a *product* of importance weights; multiplying the importance weights like this can result in high variance, particularly on long time horizons. One common solution is to use weighted importance sampling or to truncate the weights at some threshold N (Equation 6.8), which introduces bias but can dramatically reduce variance, often improving finite-sample performance.

$$\hat{\omega}_j^{\text{trunc}} = \min\left(N, \frac{\pi(a_j|x_{o,j})}{\pi_b(a_j|x_{o,j})}\right) \tag{6.8}$$

The choice of truncation threshold N represents a bias-variance tradeoff that must be navigated based on the specific application and available sample size.

We demonstrate the doubly robust estimator with a minimal Python implementation. First, we define a function that directly implements Equation 6.4, accepting pre-computed propensity scores and outcome predictions.

Doubly Robust Estimation in Python

```
import numpy as np

def doubly_robust_ate(R, A, pi_hat, Q_hat_1, Q_hat_0):
    """
    Doubly robust estimator for the Average Treatment Effect.

    Parameters
    ----------
    R : array of shape (n,)
        Observed rewards r_j
    A : array of shape (n,)
        Observed actions a_j in {0, 1}
    pi_hat : array of shape (n,)
        Estimated propensity scores pi_hat(a=1|x_o)
    Q_hat_1 : array of shape (n,)
        Estimated outcome E[R|x_o, a=1]
    Q_hat_0 : array of shape (n,)
        Estimated outcome E[R|x_o, a=0]

    Returns
    -------
    tau_dr : float
        Doubly robust ATE estimate E[R|do(a=1)] - E[R|do(a=0)]
    """
```

```
    # Clip propensity scores to avoid extreme weights (Equation 6.8)
    pi_clipped = np.clip(pi_hat, 0.01, 0.99)

    # Importance weights omega_hat_j
    omega_1 = 1 / pi_clipped
    omega_0 = 1 / (1 - pi_clipped)



    # Observed Q value
    Q_hat_obs = np.where(A == 1, Q_hat_1, Q_hat_0)

    # Doubly robust components (Equation 6.4)
    # Residual term: omega_hat_j * (r_j - Q_hat(x_{o,j}, a_j))
    residual_1 = A * omega_1 * (R - Q_hat_obs)
    residual_0 = (1 - A) * omega_0 * (R - Q_hat_obs)

    # Direct term: Q_hat(x_o, 1) - Q_hat(x_o, 0)
    direct_term = Q_hat_1 - Q_hat_0

    # DR estimate for each observation, then average
    tau_hat = (residual_1 - residual_0) + direct_term
    return tau_hat.mean()
```

To demonstrate the "double" in doubly robust, we simulate confounded observational data and compare estimates under correct and incorrect model specification. The confounding structure mirrors our earlier clinical examples, wherein observed patient characteristics X_o affect both treatment assignment and outcomes.

```
from sklearn.linear_model import LogisticRegression, Ridge

np.random.seed(1998)
n = 5000

# Observed covariates that confound treatment and outcome
X_o = np.random.randn(n, 2)

# Treatment depends on X_o (no unobserved confounding)
logit = 0.5 * X_o[:, 0] + 0.3 * X_o[:, 1]
pi_true = 1 / (1 + np.exp(-logit))
A = np.random.binomial(1, pi_true)

# Outcome: R = tau * A + X_o @ beta + epsilon
tau_true = 2.0
R = tau_true * A + 1.5 * X_o[:, 0] +
    0.8 * X_o[:, 1] + np.random.randn(n)

# Correct models: trained on true covariates
pi_model_correct = LogisticRegression().fit(X_o, A)
pi_hat_correct = pi_model_correct.predict_proba(X_o)[:, 1]

Q_model_correct = Ridge().fit(np.column_stack([X_o, A]), R)
Q_hat_1_correct = Q_model_correct.predict(np.column_stack([X_o,
    np.ones(n)]))
Q_hat_0_correct = Q_model_correct.predict(np.column_stack([X_o,
    np.zeros(n)]))

# Wrong models: trained on noise (ignores true confounders)
X_noise = np.random.randn(n, 2)
pi_model_wrong = LogisticRegression().fit(X_noise, A)
pi_hat_wrong = pi_model_wrong.predict_proba(X_noise)[:, 1]

Q_model_wrong = Ridge().fit(np.column_stack([X_noise, A]), R)
Q_hat_1_wrong = Q_model_wrong.predict(np.column_stack([X_noise,
    np.ones(n)]))
Q_hat_0_wrong = Q_model_wrong.predict(np.column_stack([X_noise,
    np.zeros(n)]))
```

We now compute the doubly robust estimate under four scenarios: i) both models correct, ii) only propensity correct, iii) only outcome correct, and iv) neither correct.

```
tau_both = doubly_robust_ate(R, A, pi_hat_correct,
                             Q_hat_1_correct, Q_hat_0_correct)
tau_pi_only = doubly_robust_ate(R, A, pi_hat_correct,
                                Q_hat_1_wrong, Q_hat_0_wrong)
tau_Q_only = doubly_robust_ate(R, A, pi_hat_wrong,
                               Q_hat_1_correct, Q_hat_0_correct)
tau_neither = doubly_robust_ate(R, A, pi_hat_wrong,
                                Q_hat_1_wrong, Q_hat_0_wrong)

tau_naive = R[A == 1].mean() - R[A == 0].mean()
```

Running this code yields the following output:

```
True ATE:                          2.000
Naive (confounded):                2.851
--------------------------------------------
DR (both correct):                 1.940
DR (propensity correct only):      1.939
DR (outcome correct only):         1.940
DR (neither correct):              2.851
```

These results confirm the theoretical prediction from Equation 6.5: The estimator achieves near-unbiased estimates when *either* model is correctly specified, but collapses to the naive (confounded) estimate when both models fail. The residual bias of approximately 0.06 reflects finite-sample variance rather than systematic error.

In practice, the `econml` [12] package provides a production-ready implementation with cross-fitting and confidence intervals; example code follows.

Doubly Robust Estimation with EconML

```
from econml.dr import DRLearner
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor

dr_learner = DRLearner(
    model_propensity=RandomForestClassifier(n_estimators=100,
        max_depth=5),
    model_regression=RandomForestRegressor(n_estimators=100,
        max_depth=5),
    cv=5
)
dr_learner.fit(R, A, X=X_o)
tau_hat = dr_learner.effect(X_o)
```

6.4 Nonlinear Approximation Methods

When faced with a partially observed system, POMDPs and MSMs are not our only options; another alternative is to directly model the nonlinear relationship between inputs and outputs. Consider, again, what happens when we try to apply an MDP framework to the observed part of a full POMDP: The reward r_j will depend not only on the state x_j,

but also the action a_{j-1}, and potentially previous state-action pairs as well. The domain of the function ρ expands from Ω to at least $\Omega \times \mathcal{A}$. This dependency arises from the unobserved variables x_m, through which a_{j-1} affects r_j independently of the observed state x_j. The function ρ becomes *nonlinear* in the state space Ω. When ignorability assumptions are violated, decision-making systems can converge to suboptimal policies.

The Bayesian belief-state method discussed in the previous section effectively linearizes the stochastic process through estimation of the unobserved factors. However, we may not know what unobserved factors actually exist, and they might not be estimable at all. An alternative approach accepts the nonlinearity and approximates it using a first-order Taylor series expansion. This Taylor series approach to nonlinear approximation appears in the *Extended Kalman Filter*, a more robust version of the Kalman filter introduced earlier. Recall that the Kalman filter can be viewed as a special case of actionless Value Estimation wherein the MDP is linear and Gaussian [91].

For a nonlinear system described by Equations 6.9-6.10, the EKF approximates the nonlinear functions using the first-order Taylor expansions in (6.11)-(6.12).

$$X_{j+1} = f(X_j, a_j) + u_j \tag{6.9}$$

$$X_{o,j} = h(X_j) + \nu_j \tag{6.10}$$

$$F_j = \left.\frac{\partial f}{\partial X}\right|_{\hat{X}_{j|j}, a_j} \tag{6.11}$$

$$H_j = \left.\frac{\partial h}{\partial X}\right|_{\hat{X}_{j|j-1}} \tag{6.12}$$

The assumption underlying this approximation is that higher-order terms of the Taylor series expansion may not substantially influence decision-making. We might say they are *relatively ignorable* with respect to the decision function π. We will return to this concept in the discussion about relative ignorability.

6.5 Further Reading

The work produced in [106, 165, 38] provides a framework for identifying causal effects even with unobserved confounding through the use of bridge functions that link observational and interventional distributions. [94] and [156] show that causal effects can be identified from partially observed data when the causal structure satisfies certain graphical properties, providing formal conditions for when POMDP belief updates and MSM weights can correctly identify causal effects.

Bareinboim's comprehensive reference [11] discusses the relationship between MDPs and SCMs. The treatment-confounder feedback formalized by [62] provides the sequential exchangeability conditions under which both POMDP and MSM approaches yield valid Causal Inferences.

Peters [120] also formalized a causal invariance principle, showing that causal predictors can be identified by finding predictors that lead to invariant predictions across different environments. [6] extended the invariance principle to deep learning contexts through an Invariant Risk Minimization (IRM) framework, while [189] applied similar principles specifically to Reinforcement Learning in Block MDPs.